A Full-Stack Hidden Markov Model Platform for Market Regime Detection, Distributed Ticker Analysis, and Regime-Aware Optimization

Aarush Gupta (2024AIB1174) and Arjun Aggarwal (2024AIB1289)
HMM Market Regime Detection Platform
AI111: Mathematical Foundations of AI and Data Engineering

Abstract? This report presents a complete technical study of a full-stack market regime detection platform based on Hidden Markov Models (HMMs). The project combines a reusable Python machine learning package, a FastAPI backend, a React and Plotly frontend, Docker-based containerization, Ray-based distributed execution, OR-Tools optimization, and Kubernetes deployment manifests. The core analytical task is the identification of latent financial market regimes from stock return observations. The platform supports direct OHLCV input, ticker-based data acquisition through Yahoo Finance, batch analysis of multiple securities, and a simple regime-aware stock allocation optimization workflow. This report describes the mathematical formulation of the HMM, the preprocessing and inference pipeline, the backend API architecture, the frontend visualization layer, the distributed processing layer, the optimization layer, and the deployment architecture. The system is intentionally educational and modular, emphasizing separation of concerns, reproducibility, and extensibility across machine learning, web services, and cloud-native development layers.

Index Terms? Hidden Markov Model, market regime detection, FastAPI, React, Plotly, Ray, OR-Tools, Docker, Kubernetes, financial machine learning, distributed computing.

INTRODUCTION

Financial markets alternate between latent behavioral phases such as low-volatility bull markets, high-volatility selloffs, and neutral consolidation periods. These phases are not directly observed in raw price data, but they influence observable quantities such as returns, volatility, drawdowns, and volume. A market regime detection system attempts to infer these hidden states from historical observations and convert them into interpretable labels that support analysis, visualization, and downstream decision making.

This project implements a complete market regime detection platform centered on a Gaussian Hidden Markov Model. The system is not limited to a standalone notebook or script. Instead, it is organized as a production-style full-stack application with a reusable Python package, a web API, a browser dashboard, distributed task execution, integer optimization, and deployment artifacts. The project therefore covers both the mathematical modeling layer and the software engineering layer required to expose the model as an interactive analytical service.

The main contributions of the system are as follows:

? A modular Python package for loading market data, preprocessing returns, training an HMM, performing inference, and generating visualizations.

? A reusable inference function, predict_market_regime(data), that returns hidden states, transition probabilities, and predicted regime labels.

? A FastAPI backend exposing health, direct prediction, ticker prediction, batch prediction, and optimization endpoints.

? A React dashboard that allows users to enter tickers and dates, view regime outputs, inspect transition probabilities, and run a simple OR-Tools allocation problem.

? Ray integration for distributed ticker analysis and optimization tasks, with graceful fallback to sequential execution.

? Docker, Docker Compose, and Kubernetes manifests for reproducible local and containerized deployment.

SYSTEM OVERVIEW

The platform is organized into several layers. The machine learning layer is implemented in the market_regime package. The backend layer is implemented in the app package using FastAPI, Pydantic schemas, service modules, Ray task wrappers, and optimization modules. The frontend layer is implemented in React with Vite, Plotly.js, and reusable chart components. The deployment layer contains Dockerfiles, Docker Compose configuration, Kubernetes Deployments, and Kubernetes Services.

At a high level, the system follows the workflow:

1. The user enters a ticker symbol and a date range in the frontend.
2. The frontend sends a request to the FastAPI backend.
3. The backend downloads historical OHLCV data using Yahoo Finance.
4. The preprocessing module computes returns and constructs the observation sequence.
5. The HMM model is trained or reused.
6. Viterbi decoding produces the most likely hidden state sequence.
7. Regime labels are assigned using state-level return and volatility properties.
8. The backend returns dates, close prices, returns, hidden states, transition probabilities, and regime labels.
9. The frontend renders charts, timelines, heatmaps, distributions, and optimization controls.

MACHINE LEARNING FORMULATION

Hidden Markov Model Definition

The core model is a Gaussian Hidden Markov Model. Let the observed market sequence be

$$O = \O_1, O_2, , O_T\,$$

where each observation is derived primarily from market returns. Let the hidden state sequence be

$$S = \S_1, S_2, , S_T\, S_t \1,2,,N\.$$

Each hidden state represents an unobserved market regime. The model is parameterized by

$$= (, A,),$$

where is the initial state distribution, A is the state transition matrix, and represents emission distribution parameters.

The first-order Markov assumption is

$$P(S_t=j S_t-1=i, S_t-2,) = P(S_t=j S_t-1=i)=a_ij.$$

The transition matrix satisfies

$$_j=1^N a_ij=1, a_ij 0.$$

Gaussian Emission Model

For a univariate return observation, the emission distribution for state i is modeled as

$$O_t S_t=i N(_i, _i^2).$$

The probability density of observing O_t in state i is

$$b_i(O_t)=12_i^2 (-(O_t-_i)^22_i^2).$$

The learned state means and variances are then used for both probabilistic inference and regime interpretation.

Forward--Backward Inference

The forward probability is defined as

$$_t(i)=P(O_1,O_2,,O_t,S_t=i).$$

It is recursively computed as

$$_t(j)=b_j(O_t)_i=1^N_t-1(i)a_ij.$$

The backward probability is

$$_t(i)=P(O_t+1,O_t+2,,O_T S_t=i,),$$

with recursion

$$_t(i)=_j=1^Na_ijb_j(O_t+1)_t+1(j).$$

The posterior probability of state i at time t is

$$_t(i)= _t(i)_t(i) _k=1^N_t(k)_t(k).$$

Viterbi Decoding

The system uses Viterbi decoding to obtain the most likely hidden state path. The objective is

$$S_1:T = _S_1:T P(S_1:T O_1:T,).$$

For numerical stability, the implementation performs path comparison in log probability space. This avoids underflow when multiplying many small transition and emission probabilities over long financial time series.

Baum--Welch Parameter Estimation

When a model is not supplied, the system trains an HMM using an Expectation-Maximization procedure equivalent to Baum--Welch learning. The E-step computes state and transition posteriors. The M-step updates the initial distribution, transition probabilities, state means, and state variances to maximize expected complete-data log-likelihood. Training stops when the improvement in likelihood falls below the configured tolerance or when the iteration limit is reached.

DATA PIPELINE

Data Sources

The backend supports two main forms of input. First, users may submit OHLCV records directly as JSON. Second, users may submit a ticker symbol and date range. For ticker-based prediction, the backend downloads historical daily market data using Yahoo Finance through the yfinance library. The service validates empty responses, missing close prices, invalid tickers, and failed downloads.

Preprocessing

Raw OHLCV data is converted into a feature frame. The primary modeling feature is the return series. For a close price sequence P_t, the return can be expressed as either a simple return or log return. In the log-return case,

$$r_t=(P_t)-(P_{t-1}).$$

The preprocessing layer removes invalid rows introduced by differencing and ensures that the observation sequence is numeric and ordered. This makes the HMM independent of the original data source and allows the same inference API to accept user-provided records or downloaded ticker data.

Regime Labeling

The HMM returns numerical hidden states. Since state indices have no intrinsic financial meaning, the inference layer maps states to business-readable labels. The labeling process considers state volatility and average return. The lowest volatility state is labeled as low volatility, the highest volatility state is labeled as high volatility, and intermediate states receive neutral volatility labels. The sign of the mean return determines whether the regime is interpreted as bull or bear. This produces labels such as low volatility bull, neutral volatility bear, and high volatility bear.

REUSABLE PYTHON PACKAGE

The market_regime package is the analytical core of the project. It separates responsibilities into dedicated modules:

? data_loading: CSV and Yahoo Finance loading helpers.

? preprocessing: return calculation and observation sequence conversion.

? model_training: HMM fitting and training result objects.

? hmm: Gaussian HMM implementation and probability routines.

? inference: prediction orchestration and label assignment.

? visualization: Plotly visualization helpers.

The central reusable interface is predict_market_regime(data). Conceptually, this function performs observation extraction, model fitting when necessary, Viterbi state decoding, posterior state probability estimation, transition matrix extraction, and regime label generation. It returns a structured prediction object containing hidden states, transition probabilities, predicted regime labels, state probabilities, and the fitted model.

BACKEND ARCHITECTURE

FastAPI Application Structure

The backend is implemented using FastAPI. It is divided into routes, schemas, services, Ray tasks, and optimization modules. This separation keeps HTTP concerns distinct from business logic and mathematical computation.

The main endpoint categories are:

? Health endpoint for readiness checks.

? Direct prediction endpoint for user-supplied OHLCV data.

? Ticker prediction endpoint for automated market data download and inference.

? Batch prediction endpoint for multiple tickers.

? Optimization endpoint for regime-aware stock allocation.

Pydantic Schemas

Pydantic models define the API boundary. Request schemas validate ticker symbols, date ranges, OHLCV records, HMM parameters, optimization assets, and optimization scenarios. Response schemas enforce consistent outputs such as hidden states, transition probabilities, regime labels, close prices, returns, dates, and optimization allocations. This provides type safety and clear API contracts between frontend and backend.

Service Layer

The service layer contains the application logic. The regime service converts request payloads into pandas frames, adds return features, invokes the reusable HMM inference function, and maps results back into response models. The ticker service handles historical data download and guards against invalid or empty datasets. The batch service orchestrates multiple ticker analyses and chooses between Ray-based distributed execution and sequential fallback.

Error Handling

The backend defines domain-specific errors that map to HTTP status codes. Examples include invalid tickers, failed downloads, empty datasets, and insufficient rows to compute returns. This prevents low-level exceptions from leaking into the API response and allows the frontend to display user-readable messages.

DISTRIBUTED COMPUTING WITH RAY

Motivation

Batch ticker analysis is embarrassingly parallel because each ticker can be downloaded, preprocessed, modeled, and decoded independently. Ray is integrated to distribute these independent tasks across workers when available.

Ray Initialization and Fallback

The backend initializes Ray during application startup. If Ray fails to initialize, the application remains available and logs the failure. This graceful fallback is important in constrained container environments where shared memory or process startup limits may prevent Ray from starting. The batch prediction and optimization services therefore check Ray readiness before dispatching remote tasks.

Remote Tasks

The Ray task layer wraps existing service functions rather than duplicating model logic. This design ensures that local and distributed execution use the same preprocessing, HMM inference, validation, and response construction paths. Ray is used for ticker analysis tasks, market regime prediction tasks, batch processing, and optimization tasks.

Fault Isolation

Batch prediction collects per-ticker results. If one ticker fails due to an invalid symbol or data download error, the failure is recorded for that ticker without terminating the entire batch request. The response reports both success and failure counts. This is important for practical financial workflows where ticker universes often contain stale, delisted, or malformed symbols.

OR-TOOLS OPTIMIZATION LAYER

Optimization Objective

The project includes a simple educational integer optimization problem using OR-Tools. Each asset has a price, expected profit, risk score, and regime label. The decision variable is the integer number of units allocated to each asset.

Let x_i be the number of units allocated to asset i. The objective is

$$\_i=1\^M p\_i x\_i,$$

where p_i is the expected profit contribution of asset i.

Constraints

The optimization is subject to a budget constraint

$$\_i=1\^M c\_i x\_i B,$$

where c_i is the asset price and B is the maximum budget.

It also includes a risk constraint

$$\_i=1\^M q\_i x\_i R,$$

where q_i is the asset risk score and R is the maximum allowed risk.

Finally, each asset has a position limit

$$0 x\_i U, x\_i Z.$$

Architecture

The intended architecture is:

FastAPI Ray Worker OR-Tools Solver Response.

If Ray is unavailable, the same solver runs sequentially. The optimization result reports solver status, objective value, total cost, total risk, and per-asset allocations.

FRONTEND ARCHITECTURE

React and Vite

The frontend is implemented using React with Vite. Functional components and hooks manage state, form input, loading states, error states, active chart selection, and API interactions. Vite provides fast local development and production bundling.

Service Separation

Frontend API logic is separated into service modules. The regime API service calls ticker prediction endpoints, while the optimization API service calls the optimization endpoint. This prevents components from directly embedding request details and makes the interface easier to maintain.

Visualization

The dashboard uses Plotly.js through React Plotly components. It provides the following visual outputs:

? Stock price history.

? Regime-colored price chart.

? Hidden state timeline.

? Returns chart.

? Transition probability heatmap.

? Regime distribution chart.

The transition probability heatmap uses categorical state axes to prevent time-series axis inference. It displays the transition matrix as a probability grid where each cell corresponds to P(S_t=j S_t-1=i).

User Interface Design

The current frontend uses a financial charting-terminal layout. The interface contains a top quote strip, a left input and output rail, a tabbed chart toolbar, legend chips, a main chart canvas, and an optimization view. The design goal is to preserve all application functionality while presenting it in a compact market-analysis format.

CONTAINERIZATION

Backend Docker Image

The backend Docker image uses a slim Python runtime. It installs backend dependencies, copies the FastAPI application and market regime package, exposes the backend port, and runs Uvicorn. Environment variables configure the host, port, and CORS origins. The backend image is optimized by separating dependency installation from application copy operations, which improves Docker layer reuse.

Frontend Docker Image

The frontend Docker image uses a multi-stage build. The first stage installs Node dependencies and builds the React production bundle. The second stage uses Nginx to serve static frontend assets. A runtime configuration script supports environment-based API URL configuration. Nginx also proxies frontend API requests to the backend service in containerized deployment.

Docker Compose

Docker Compose starts both backend and frontend services. The frontend depends on a healthy backend service and exposes the application on port 3000. The backend exposes port 8000 and includes a health check. The configuration also provides shared memory for backend execution, which is relevant for Ray workloads.

KUBERNETES DEPLOYMENT

The Kubernetes support is intentionally minimal and educational. It includes deployments for backend and frontend, replicas, labels, selectors, container ports, and resource requests and limits. NodePort services expose the backend and frontend. The frontend communicates with the backend through the Kubernetes service name, allowing internal cluster networking to route requests.

The Kubernetes manifests demonstrate:

? Containerized application deployment.

? Replica-based availability.

? Service discovery through labels and selectors.

? CPU and memory resource requests and limits.

? Minikube-based local cluster execution.

API SURFACE

The backend exposes a set of endpoints aligned with the platform workflow. The health endpoint validates service availability. The direct prediction endpoint accepts OHLCV JSON data. The ticker prediction endpoint fetches data automatically. The batch prediction endpoint parallelizes multiple ticker analyses. The optimization endpoint solves the allocation problem. Together, these endpoints expose the model as a reusable analytical service rather than a notebook-only artifact.

SECURITY AND ROBUSTNESS CONSIDERATIONS

The platform applies schema validation at the API boundary and structured error handling in service modules. Invalid tickers, missing close prices, empty downloads, and failed external data calls are converted into controlled responses. The Dockerized architecture isolates runtime dependencies. The Ray integration includes fallback behavior so the service remains available even when distributed execution is not possible. Kubernetes resource limits prevent pods from consuming unbounded CPU or memory.

LIMITATIONS

The current model uses a simple Gaussian HMM and primarily relies on returns as the observation sequence. This makes the system interpretable and educational but limits the feature richness available to the model. Regime labels are heuristic and based on volatility ordering and return sign. The OR-Tools optimization problem is intentionally simplified and does not model covariance, transaction costs, slippage, portfolio diversification, or real-world constraints. Ray is integrated for parallelism, but local container environments may require additional shared memory tuning for stable distributed execution.

FUTURE WORK

Future improvements could include multivariate HMM emissions, rolling-window retraining, model persistence, richer technical indicators, probability-calibrated regime confidence, portfolio-level risk modeling, authentication, persistent job tracking, asynchronous background jobs, enhanced Kubernetes probes, and CI/CD automation. The frontend could further support side-by-side ticker comparison, downloadable reports, and dynamic chart overlays.

CONCLUSION

This project demonstrates a complete end-to-end market regime detection platform. The mathematical core uses a Gaussian Hidden Markov Model to infer latent market states from return observations. The software architecture exposes this model through a modular Python package, a FastAPI service layer, a React visualization frontend, Ray-based distributed execution, OR-Tools optimization, Docker containers, and Kubernetes manifests. The result is both an educational study of HMM-based financial modeling and a practical demonstration of how machine learning systems can be packaged, served, visualized, scaled, optimized, and deployed.

REFERENCES

[1] L. R. Rabiner, ?A tutorial on hidden Markov models and selected applications in speech recognition,? Proceedings of the IEEE, 1989.

[2] R. S. Tsay, Analysis of Financial Time Series, Wiley, 2010.

[3] FastAPI, Ray, OR-Tools, React, Docker, and Kubernetes official documentation.